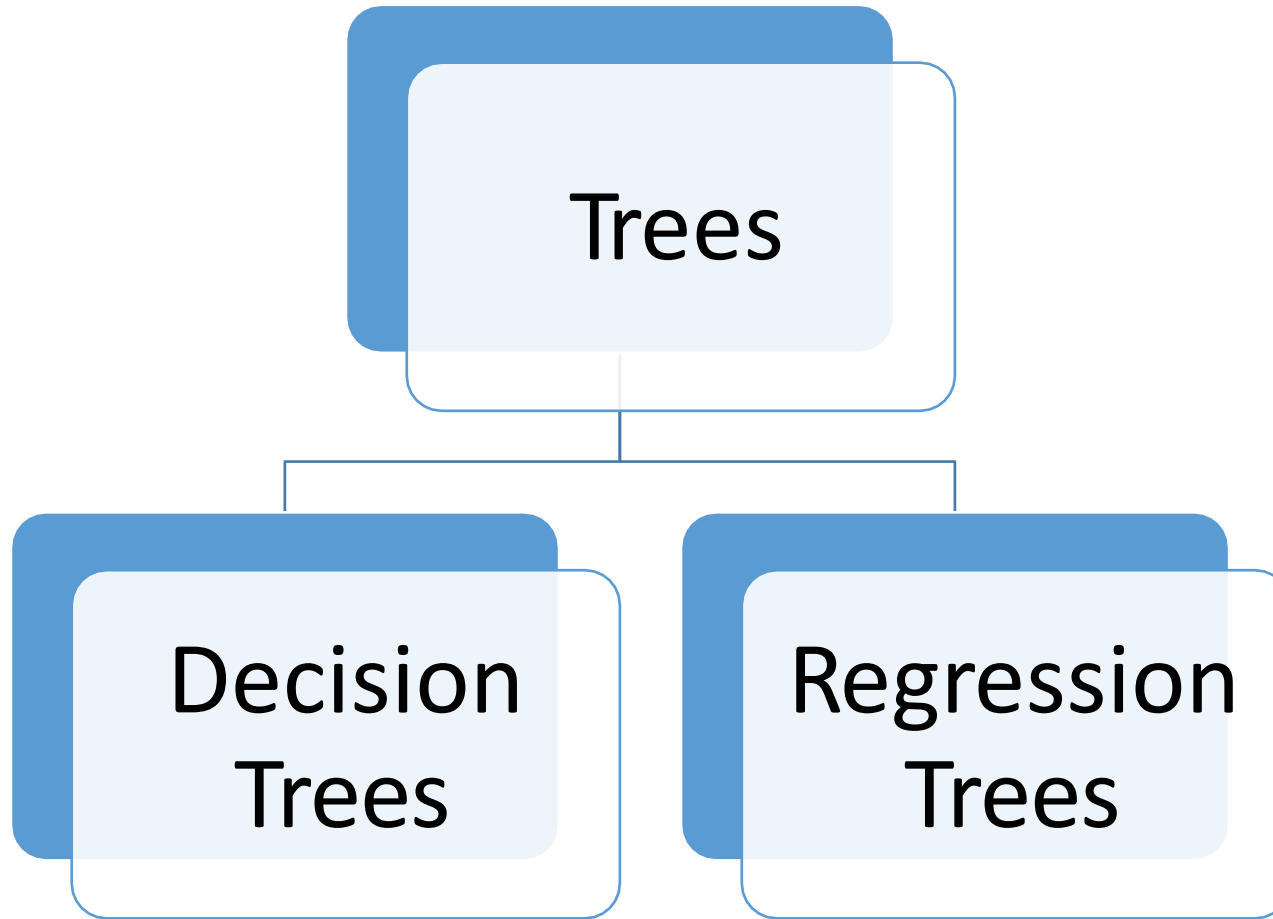


Trees

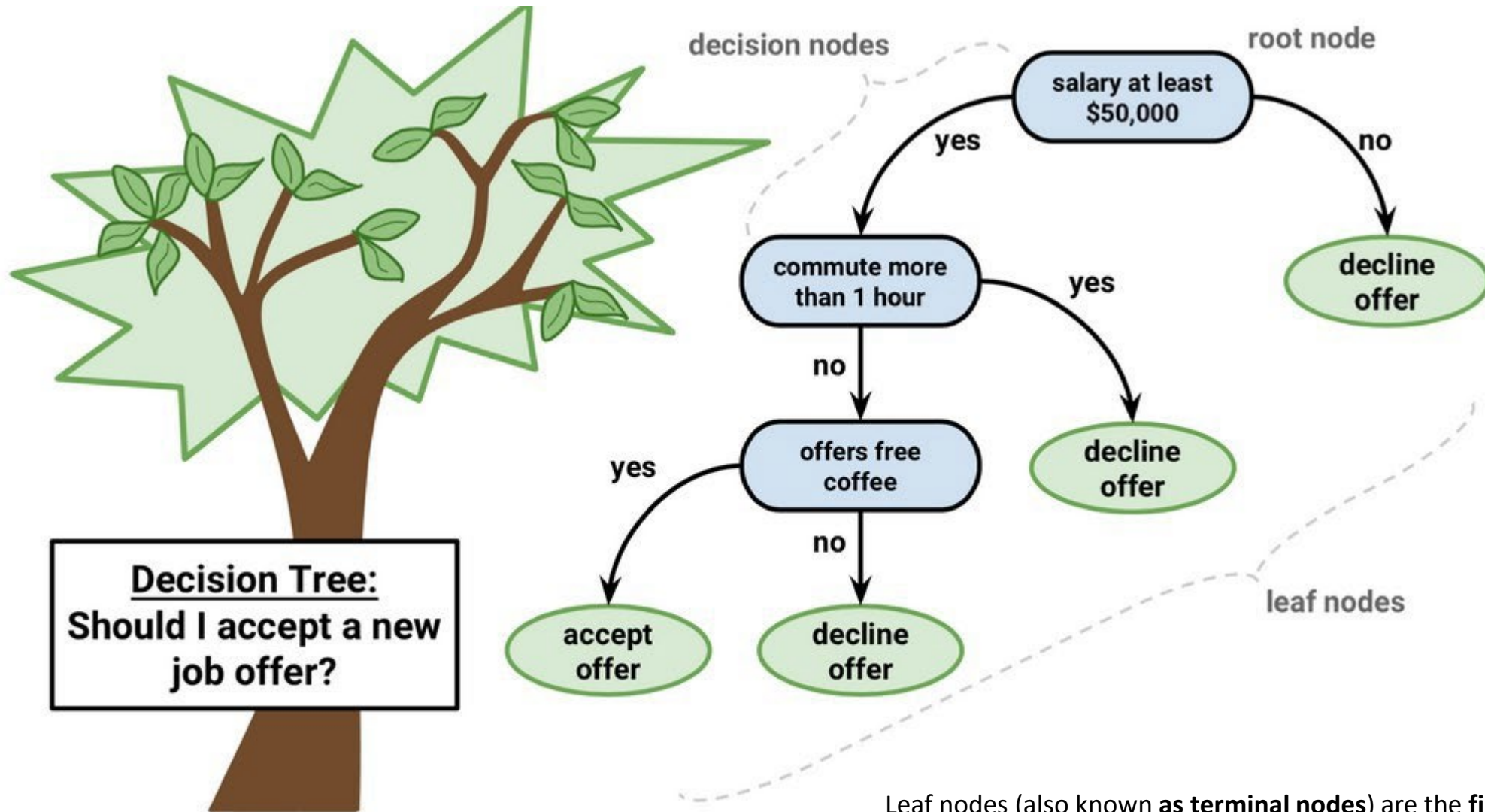
Trees



Understanding Decision Trees

- Decision tree learners are powerful classifiers that utilize a tree structure to model the relationships among the features and the potential outcomes.
- This structure earned its name due to the fact that it mirrors the way a literal tree begins at a wide trunk and splits into narrower and narrower branches as it is followed upward. In much the same way, a decision tree classifier uses a structure of branching decisions that channel examples into a final predicted class value.
- To better understand how this works in practice, let's consider the following tree, which predicts whether a job offer should be accepted. A job offer under consideration begins at the root node, where it is then passed through decision nodes that require choices to be made based on the attributes of the job. These choices split the data across branches that indicate potential outcomes of a decision. They are depicted here as yes or no outcomes, but in other cases, there may be more than two possibilities.
- If a final decision can be made, the tree is terminated by leaf nodes (also known as terminal nodes) that denote the action to be taken as the result of the series of decisions. In the case of a predictive model, the leaf nodes provide the expected result given the series of events in the tree

Understanding Decision Trees, *cont'd*



Leaf nodes (also known as **terminal nodes**) are the **final stops** in the tree, giving the **action or prediction** after all splits on that path.

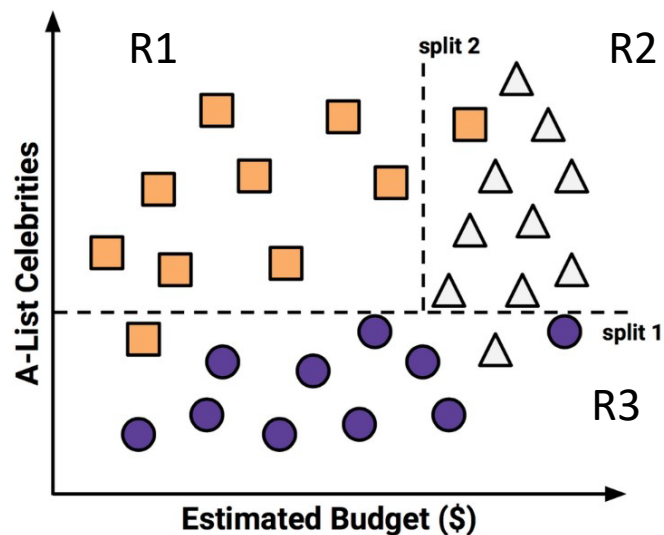
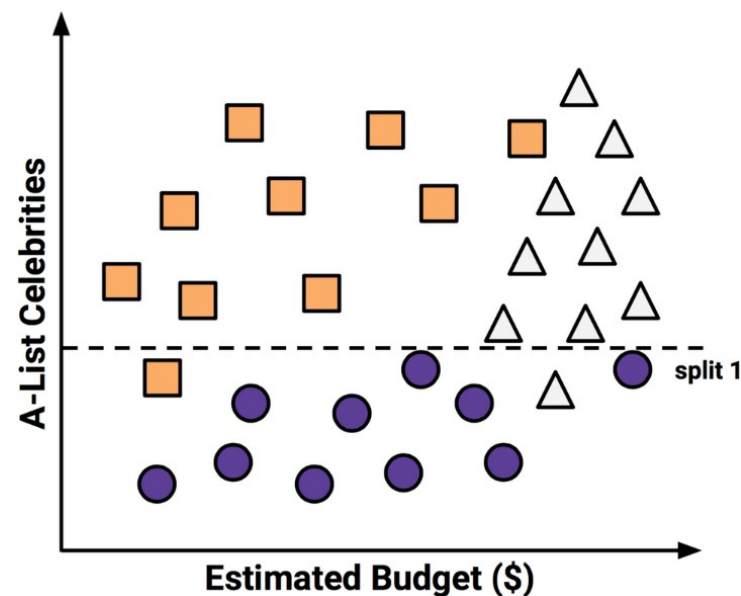
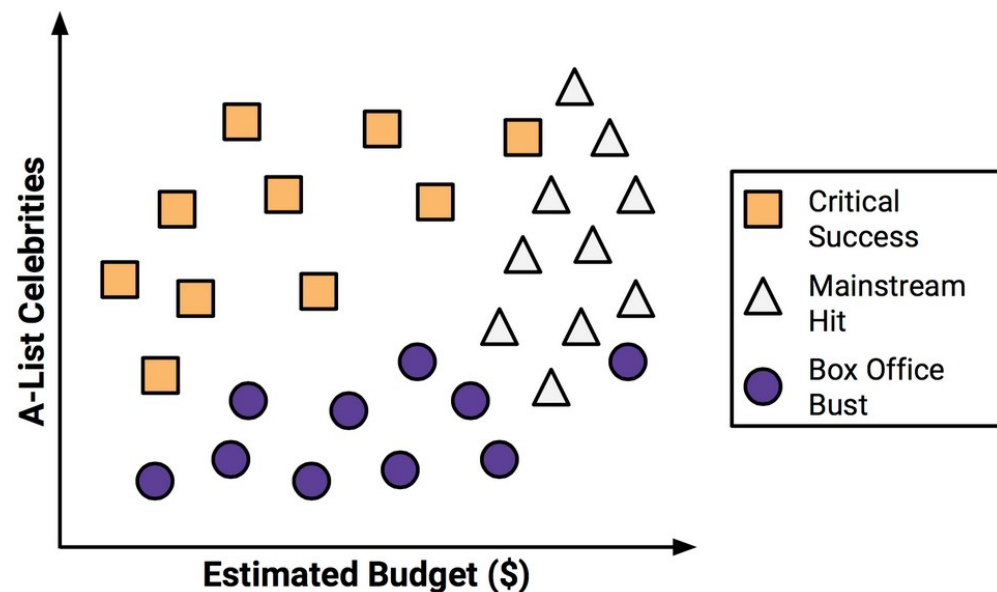
Understanding Decision Trees, *cont'd*

- A great benefit of decision tree algorithms is that the flowchart-like tree structure is not only for the machine's internal use. After the model is created, many decision tree algorithms output the resulting structure in a human-readable format. This provides insight into how and why the model works or doesn't work well for a particular task. This also makes decision trees particularly appropriate for applications in which the classification mechanism needs to be transparent for legal reasons, or if the results need to be shared with others to inform future business practices. With this in mind, some potential uses include:
 - Credit scoring models in which the criteria that cause an applicant to be rejected need to be clearly documented and free from bias
 - Marketing studies of customer behavior, such as satisfaction or churn, which will be shared with management or advertising agencies
 - Diagnosis of medical conditions based on laboratory measurements, symptoms, or rate of disease progression
- Although the previous applications illustrate the value of trees for informing decision processes, this is not to suggest that their utility ends here. In fact, decision trees are perhaps the single most widely used machine learning technique, and can be applied for modeling almost any type of data—often with excellent out-of-the-box performance.
- That said, in spite of their wide applicability, it is worth noting that there are some scenarios where trees may not be an ideal fit. This includes tasks where the data has a large number of nominal features with many levels or a large number of numeric features. These cases may result in a very large number of decisions and an overly complex tree. They may also contribute to the tendency of decision trees to overfit data, though as we will soon see, even this weakness can be overcome by adjusting some simple parameters.

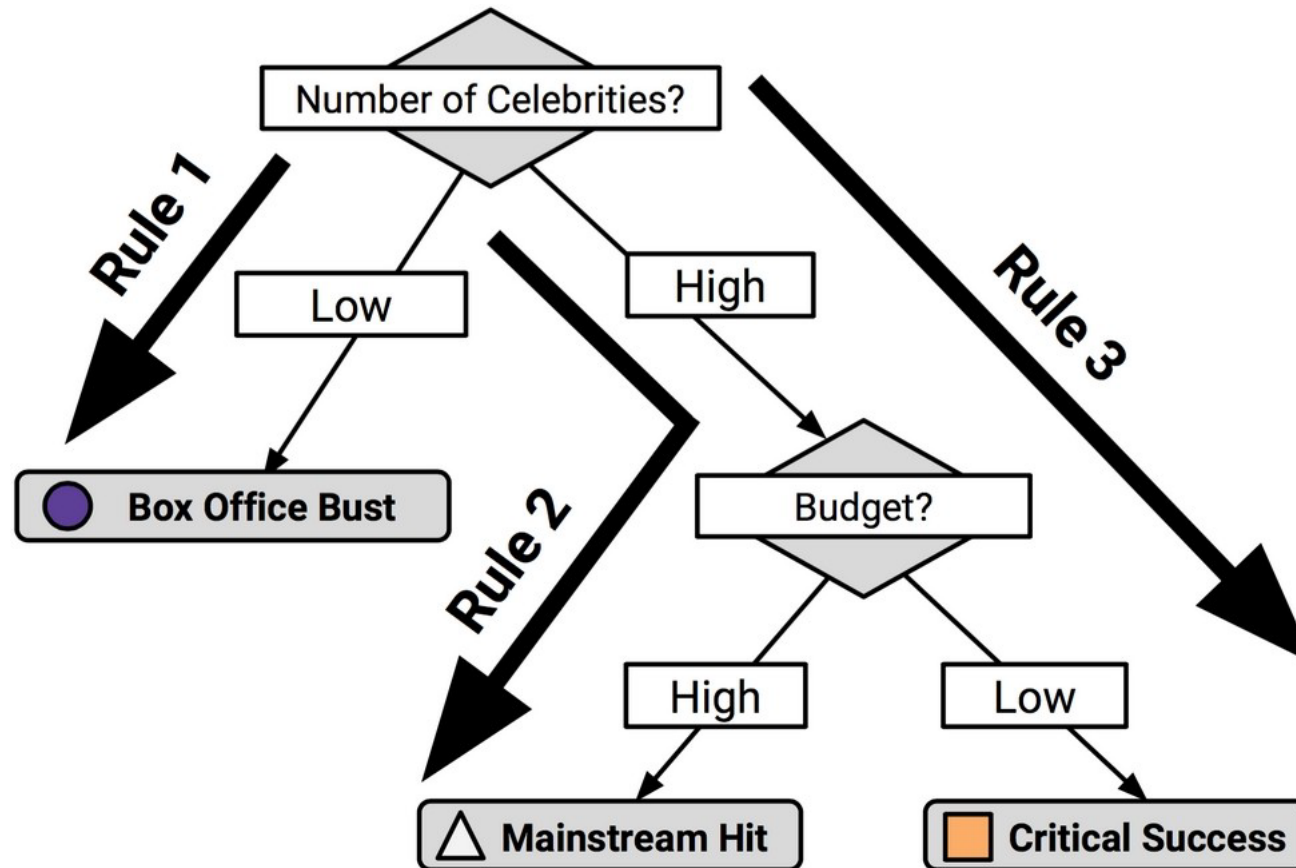
Understanding Decision Trees, *cont'd*

- Decision trees are built using a heuristic called recursive partitioning. This approach is also commonly known as divide and conquer because it splits the data into subsets, which are then split repeatedly into even smaller subsets, and so on and so forth until the process stops when the algorithm determines the data within the subsets are sufficiently homogenous, or another stopping criterion has been met.
- To see how splitting a dataset can create a decision tree, imagine a bare root node that will grow into a mature tree. At first, the root node represents the entire dataset, since no splitting has transpired. Here, the decision tree algorithm must choose a feature to split upon; ideally, it chooses the feature most predictive of the target class. The examples are then partitioned into groups according to the distinct values of this feature, and the first set of tree branches is formed.
- Working down each branch, the algorithm continues to divide and conquer the data, choosing the best candidate feature each time to create another decision node until a stopping criterion is reached. Divide and conquer might stop at a node if:
 - All (or nearly all) of the examples at the node have the same class
 - There are no remaining features to distinguish among examples
 - The tree has grown to a predefined size limit
- To illustrate the tree-building process, let's consider a simple example. Imagine that you work for a Hollywood studio, where your role is to decide whether the studio should move forward with producing the screenplays pitched by promising new authors. After returning from a vacation, your desk is piled high with proposals. Without the time to read each proposal cover-to-cover, you decide to develop a decision tree algorithm to predict whether a potential movie would fall into one of three categories: Critical Success, Mainstream Hit, or Box Office Bust.
- To build the decision tree, you turn to the studio archives to examine the factors leading to the success and failure of the company's 30 most recent releases. You quickly notice a relationship between the film's estimated shooting budget, the number of A-list celebrities lined up for starring roles, and the level of success. Excited about this finding, you produce a scatterplot to illustrate the pattern:

Understanding Decision Trees, *cont'd*

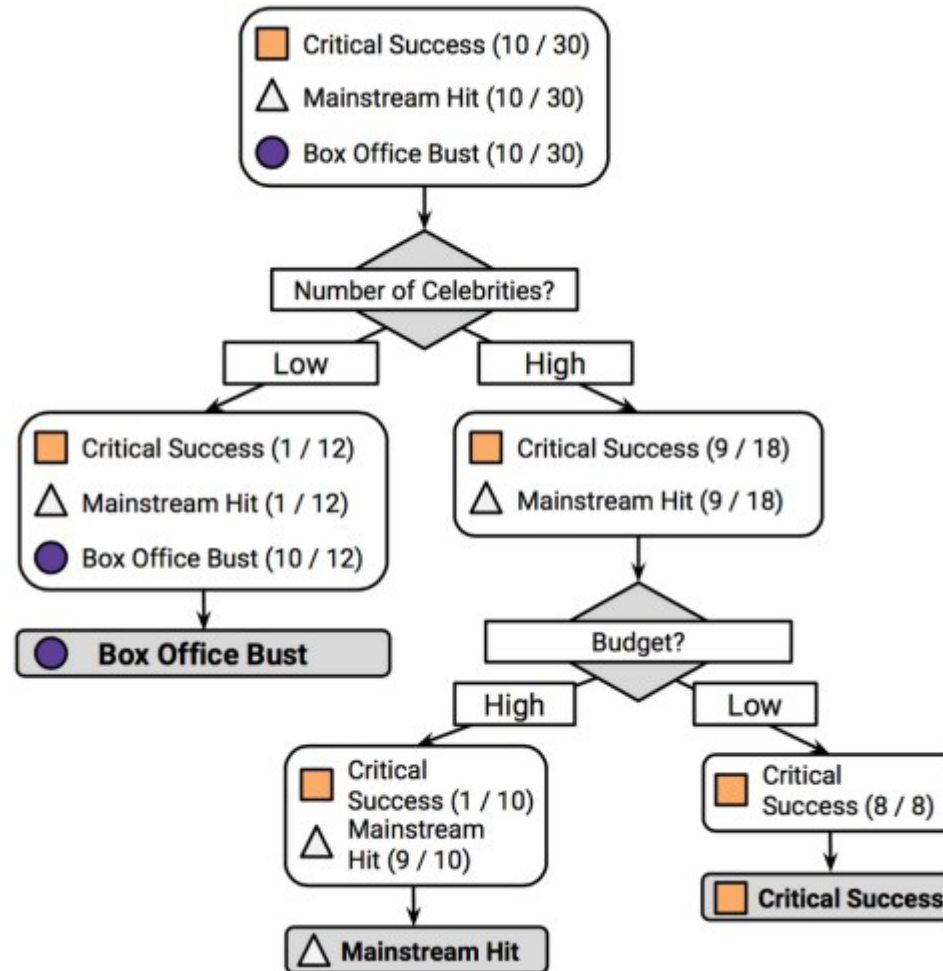


Understanding Decision Trees, *cont'd*



1. If the number of celebrities is low, then the movie will be a **Box Office Bust**.
2. If the number of celebrities is high and the budget is high, then the movie will be a **Mainstream Hit**.
3. If the number of celebrities is high and the budget is low, then the movie will be a **Critical Success**.

Understanding Decision Trees, *cont'd*



Understanding Decision Trees, *cont'd*

Strengths	Weaknesses
• An all-purpose classifier that does well on many types of problems • Highly automatic learning process, which can handle numeric or nominal features, as well as missing data • Excludes unimportant features • Can be used on both small and large datasets • Results in a model that can be interpreted without a mathematical background (for relatively small trees) • More efficient than other complex models	• Decision tree models are often biased toward splits on features having a large number of levels • It is easy to overfit or underfit the model • Can have trouble modeling some relationships due to reliance on axis-parallel splits • Small changes in training data can result in large changes to decision logic • Large trees can be difficult to interpret and the decisions they make may seem counterintuitive

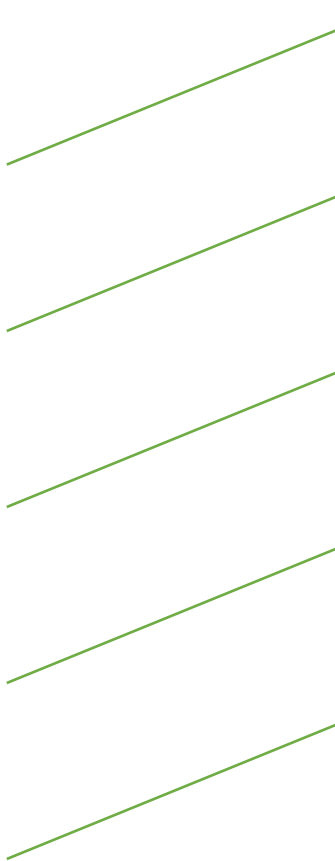
Choosing the best split

- The first challenge that a decision tree will face is to identify which feature to split upon. In the previous example, we looked for a way to split the data such that the resulting partitions contained examples primarily of a single class. The degree to which a subset of examples contains only a single class is known as **purity**, and any subset composed of only a single class is called **pure**.
- There are various measurements of purity that can be used to identify the best decision tree splitting candidate. C5.0 (R library) uses entropy, a concept borrowed from information theory that quantifies the randomness, or disorder, within a set of class values. Sets with high entropy are very diverse and provide little information about other items that may also belong in the set, as there is no apparent commonality. The decision tree hopes to find splits that reduce entropy, ultimately increasing homogeneity within the groups.
- Typically, entropy is measured in bits. If there are only two possible classes, entropy values can range from 0 to 1. For n classes, entropy ranges from 0 to $\log_2(n)$. In each case, the minimum value indicates that the sample is completely homogenous, while the maximum value indicates that the data are as diverse as possible, and no group has even a small plurality. In mathematical notion, entropy is specified as:

$$\text{Entropy}(S) = \sum_{i=1}^c -p_i \log_2(p_i)$$

Tree Impurity

Impurity of a tree is the *measure of homogeneity (or heterogeneity) of the data based on the classes.*



We say a data set is **pure or homogenous** if it contains only a **single class**

If a data set contains **several classes**, then we say that the data set is **impure or heterogeneous**

Measure of impurity of a node helps to decide on **how to split** a node, or **which node to split**

The measure should **be at a maximum** when a node is **equally divided amongst all classes**

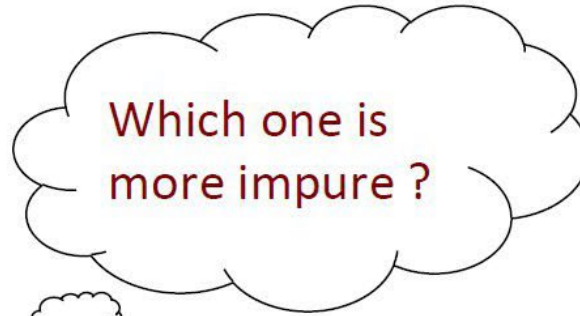
The impurity should **be minimum** if the **node is all one class**

Tree Impurity, *cont'd*



9 Cola

1 Pepsi



6 Cola

4 Pepsi



Tree Impurity, *cont'd*



9 Cola



1 Pepsi



I am pure.
I am homogeneous !!



6 Cola



4 Pepsi



I am Impure !!
Decision Trees, help me !!!

Measure of Impurity

Entropy

$$entropy(D) = - \sum_{j=1}^{|C|} \Pr(c_j) \log_2 \Pr(c_j)$$
$$\sum_{j=1}^{|C|} \Pr(c_j) = 1,$$

- ❑ Entropy of a pure data set (consisting of a single class) is zero probability ($\Pr(c_j)$ is 1 and $\log_2(1) = 0$)
- ❑ Value of entropy can be larger than 1

Gini Index

$$GiniIndex = 1 - \sum_{j=1}^{|C|} (\Pr(c_j))^2$$
$$\sum_{j=1}^{|C|} \Pr(c_j) = 1,$$

- ❑ Gini Index of a pure data set (consisting of single class) is zero probability ($\Pr(c_j)$ is 1 and $1 - (1)^2 = 0$)
- ❑ Value of Gini Index is always between 0 and 1

Classification Error

$$ClassificationError = 1 - \max(\Pr(c_j))$$

- ❑ Similar to Gini Index and Entropy, Classification Error Index of a pure table (consisting of single class) is zero because probability (p_j) is 1 and $1 - \max(1) = 0$
- ❑ Value of Classification Index is always between 0 and 1.

Attribute Selection based on Entropy Measure

Computing Entropy for three Example Data Set D

1. The data set D has 50% positive examples ($\text{Pr}(\text{positive})=0.5$) and 50% negative examples ($\text{Pr}(\text{negative})=0.5$)

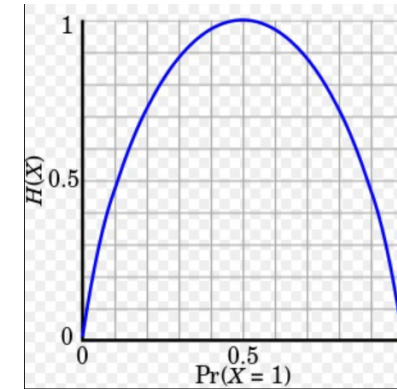
$$\text{Entropy}(D) = -0.5 \times \log_2 0.5 - 0.5 \times \log_2 0.5 = 1$$

2. The data set D has 20% positive examples ($\text{Pr}(\text{positive})=0.2$) and 80% negative examples ($\text{Pr}(\text{negative})=0.8$)

$$\text{Entropy}(D) = -0.2 \times \log_2 0.2 - 0.8 \times \log_2 0.8 = 0.722$$

3. The data set D has 100% positive examples ($\text{Pr}(\text{positive})=1$) and no negative examples ($\text{Pr}(\text{negative})=0$)

$$\text{Entropy}(D) = -1 \times \log_2 1 - 0 \times \log_2 0 = 0$$



As illustrated by the peak in entropy at $x = 0.50$, a 50-50 split results in the maximum entropy. As one class increasingly dominates the other, the entropy reduces to zero.

The data set gets purer as the value of entropy tends towards the minimum!

How to decide the split

Which one is a better split....

OR?

Entropy Calculations

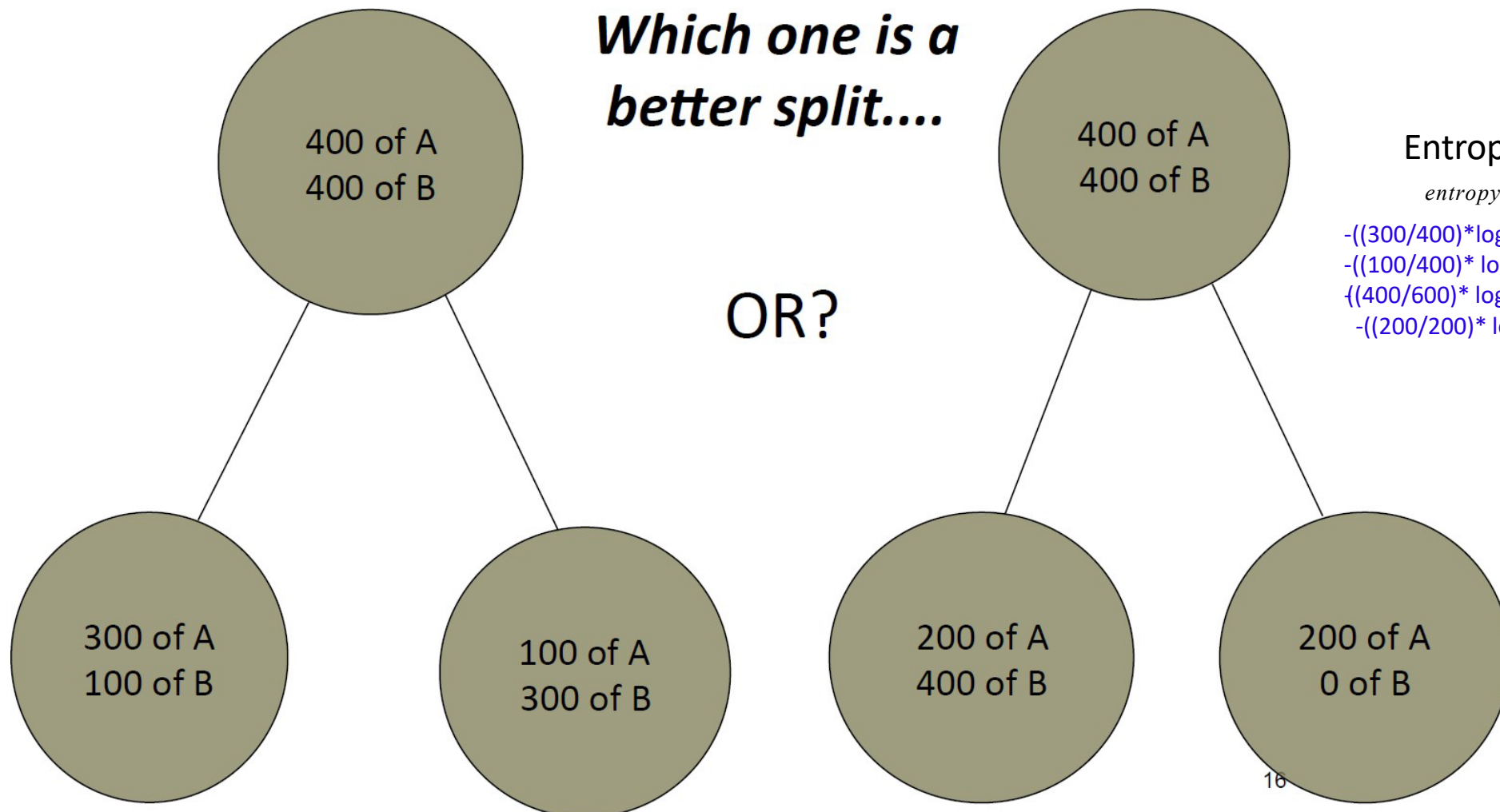
$$entropy(D) = - \sum_{j=1} \Pr(c_j) \log_2 \Pr(c_j)$$

$$-((300/400) * \log_2(300/400) + (100/400) * \log_2(100/400)) = 0.81$$

$$-((100/400) * \log_2(100/400) + (300/400) * \log_2(300/400)) = 0.81$$

$$-((400/600) * \log_2(400/600) + (200/600) * \log_2(200/600)) = 0.91$$

$$-((200/200) * \log_2(200/200) + (0/200) * \log_2(0/200)) = 0$$



18

$$-(0.75 * \log_2 0.75 + 0.25 * \log_2 0.25) = 0.81$$

$$-(0.25 * \log_2 0.25 + 0.75 * \log_2 0.75) = 0.81$$

$$-(0.33 * \log_2 0.33 + 0.67 * \log_2 0.67) = 0.91$$

$$-(1 * \log_2 1 + 0 * \log_2 0) = 0.91$$

Entropy = 0.811

Entropy = 0.811

Entropy = 0.91

Entropy = 0

Information gain

- The total entropy resulting from a split is the sum of entropy of each of the n partitions weighted by the proportion of examples falling in the partition (w_i). The expected entropy if A_i is used as the current root:

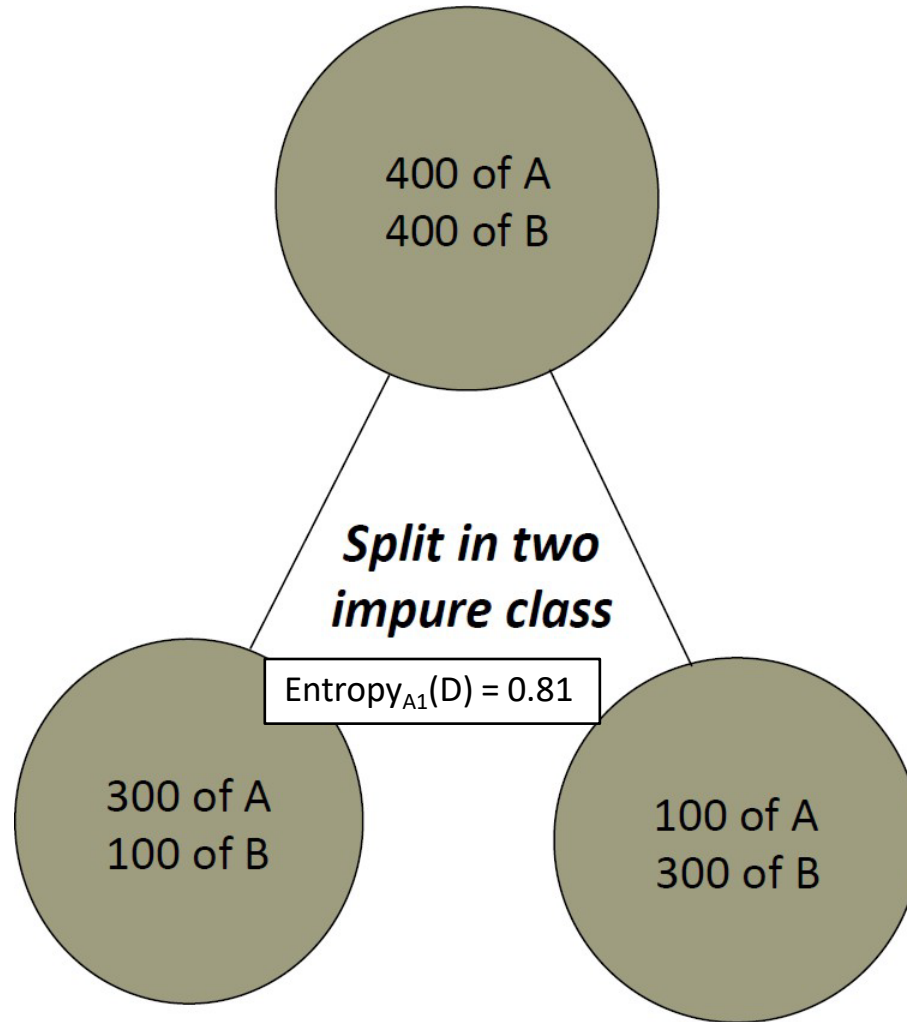
$$entropy_{A_i}(D) = \sum_{j=1}^n w_j \times entropy(D_j)$$

- The total entropy resulting from a split is the sum of entropy of each of the n partitions weighted by the proportion of examples falling in the partition (w_i).
- Information gained by selecting attribute A_i to branch or to partition the data is

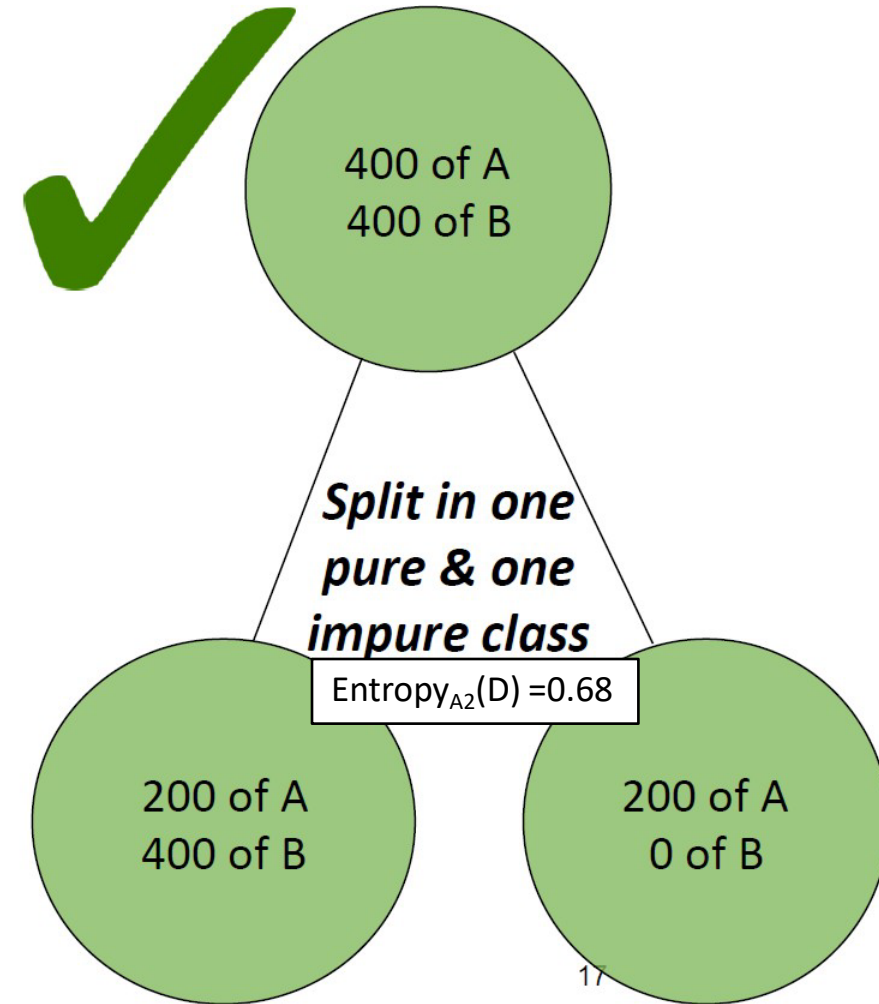
$$gain(D, A_i) = entropy(D) - entropy_{A_i}(D)$$

We choose the attribute with the highest gain to branch/split the current tree

Best Split



$$400/800 * 0.81 + 400/800 * 0.81 = 0.81$$



$$600/800 * 0.91 + 200/800 * 0 = 0.68$$

Decision Tree example with real life data

Count of Class		
No	Yes	Total
6	9	15

$$entropy(D) = -\frac{6}{15} \times \log_2 \frac{6}{15} - \frac{9}{15} \times \log_2 \frac{9}{15} = 0.971$$

$$entropy_{own_house}(D) = \frac{7}{15} \times entropy(D_1) + \frac{8}{15} \times entropy(D_2)$$

$$= \frac{7}{15} \times 0 + \frac{8}{15} \times 0.811$$

$$= 0.433$$

Count of Class			
Row Labels	No	Yes	Total
False	6	2	8
True		7	7
Total	6	9	15

$$entropy_{Age}(D) = \frac{5}{15} \times entropy(D_1) + \frac{5}{15} \times entropy(D_2) + \frac{5}{15} \times entropy(D_3)$$

$$= \frac{5}{15} \times 0.971 + \frac{5}{15} \times 0.971 + \frac{5}{15} \times 0.722$$

$$= 0.888$$

Row Labels	No	Yes	Total
Middle	2	3	5
Old	1	4	5
Young	3	2	5
Total	6	9	15

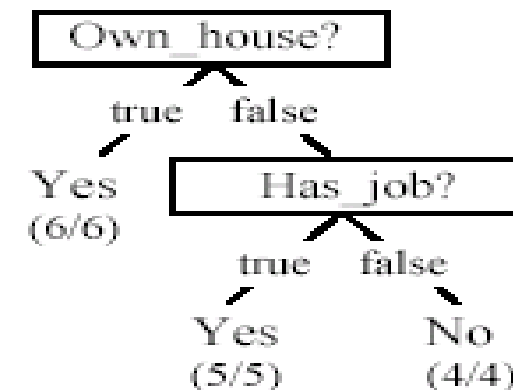
$$gain(D, Age) = 0.971 - 0.888 = 0.083$$

$$gain(D, own_house) = 0.971 - 0.433 = 0.538$$

$$gain(D, Has_Job) = 0.971 - 0.647 = 0.324$$

$$gain(D, Credit_Rating) = 0.971 - 0.608 = 0.363$$

ID	Age	Has Job	Own House	Credit Rating	Class
1	Young	False	False	Fair	No
2	Young	False	False	Good	No
3	Young	True	True	Good	Yes
4	Young	True	True	Fair	Yes
5	Young	False	False	Fair	No
6	Middle	False	False	Fair	No
7	Middle	False	False	Good	No
8	Middle	True	True	Good	Yes
9	Middle	False	True	Excellent	Yes
10	Middle	False	True	Excellent	Yes
11	Old	False	True	Excellent	Yes
12	Old	False	True	Good	Yes
13	Old	True	False	Good	Yes
14	Old	True	False	Excellent	Yes
15	Old	False	False	Fair	No



Pruning the decision tree

A decision tree can continue to grow indefinitely, choosing splitting features and dividing into smaller and smaller partitions until each example is perfectly classified or the algorithm runs out of features to split on. However, if the tree grows overly large, many of the decisions it makes will be overly specific and the model will be overfitted to the training data. The process of **pruning** a decision tree involves reducing its size such that it generalizes better to unseen data.

One solution to this problem is to stop the tree from growing once it reaches a certain number of decisions or when the decision nodes contain only a small number of examples. This is called **early stopping** or **pre-pruning** the decision tree. As the tree avoids doing needless work, this is an appealing strategy. However, one downside to this approach is that there is no way to know whether the tree will miss subtle but important patterns that it would have learned had it grown to a larger size.

An alternative, called **post-pruning**, involves growing a tree that is intentionally too large and pruning leaf nodes to reduce the size of the tree to a more appropriate level. This is often a more effective approach than pre-pruning because it is quite difficult to determine the optimal depth of a decision tree without growing it first. Pruning the tree later on allows the algorithm to be certain that all of the important data structures were discovered.

Overfitting problem and resolution

- ❑ **Overfitting:** A tree may overfit the training data
 - ❑ Good accuracy on training data but poor on test data
 - ❑ Symptoms: tree too deep and too many branches, some may reflect anomalies due to noise or outliers
- ❑ Two approaches to avoid overfitting

Approach	Stopping Criteria	Advantages	Disadvantages
Pre Pruning	Halts the tree construction once it reaches a specified threshold for a parameter (eg. Tree depth, error rate, etc)	Computationally faster	May lose out on critical information
Post Pruning	The entire tree is constructed and then pruned upwards.	Captures all the critical information	Computationally intensive

R library and Example

Data with these characteristics are available in a dataset donated to the UCI Machine Learning Repository (<http://archive.ics.uci.edu/ml>) by Hans Hofmann of the University of Hamburg. The dataset contains information on loans obtained from a credit agency in Germany.

The global financial crisis of 2007-2008 highlighted the importance of transparency and rigor in banking practices. As the availability of credit was limited, banks tightened their lending systems and turned to machine learning to more accurately identify risky loans.

C5.0 decision tree syntax

using the `C5.0()` function in the `C50` package

Building the classifier:

```
m <- C5.0(train, class, trials = 1, costs = NULL)
```

- `train` is a data frame containing training data
- `class` is a factor vector with the class for each row in the training data
- `trials` is an optional number to control the number of boosting iterations (set to 1 by default)
- `costs` is an optional matrix specifying costs associated with various types of errors

The function will return a C5.0 model object that can be used to make predictions.

Making predictions:

```
p <- predict(m, test, type = "class")
```

- `m` is a model trained by the `C5.0()` function
- `test` is a data frame containing test data with the same features as the training data used to build the classifier.
- `type` is either `"class"` or `"prob"` and specifies whether the predictions should be the most probable class value or the raw predicted probabilities

The function will return a vector of predicted class values or raw predicted probabilities depending upon the value of the `type` parameter.

Example:

```
credit_model <- C5.0(credit_train, loan_default)
credit_prediction <- predict(credit_model,
                             credit_test)
```

Regression Trees

- Trees that can perform numeric prediction offer a compelling, yet often overlooked, alternative to regression modeling. The strengths and weaknesses of regression trees and model trees relative to the more common regression methods are listed in the following table

Strengths	Weaknesses
• Combines the strengths of decision trees with the ability to model numeric data • Does not require the user to specify the model in advance • Uses automatic feature selection, which allows the approach to be used with a very large number of features • May fit some types of data much better than linear regression • Does not require knowledge of statistics to interpret the model	• Not as well-known as linear regression • Requires a large amount of training data • Difficult to determine the overall net effect of individual features on the outcome • Large trees can become more difficult to interpret than a regression model

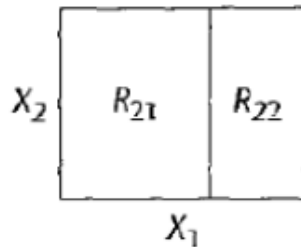
Regression Trees, cont'd

- Trees for numeric prediction are built in much the same way as they are for classification. Beginning at the root node, the data is partitioned using a divide and conquer strategy according to the feature that will result in the greatest increase in homogeneity in the outcome after a split is performed.
- In classification trees, you will recall that homogeneity is measured by entropy. This is undefined for numeric data. Instead, for numeric decision trees, homogeneity is measured by statistics such as SSE, variance, standard deviation, or absolute deviation from the mean.

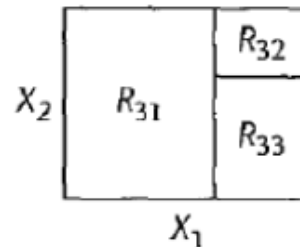
Regression Trees, cont'd

- Roughly speaking, there are two steps:
 - We divide the predictor space—that is, the set of possible values for $X_1, X_2, \dots, X_p$ into J distinct and non-overlapping regions, $R_1, R_2, \dots, R_J$.
 - For every observation that falls into the region R_j , we make the same prediction, which is simply the mean of the response values for the training observations in R_j .
- For instance, suppose that in Step 1 we obtain two regions, R_1 and R_2 , and that the response mean of the training observations in the first region is 10, while the response mean of the training observations in the second region is 20. Then for a given observation $X = x$, if $x \in R_1$ we will predict a value of 10, and if $x \in R_2$ we will predict a value of 20.

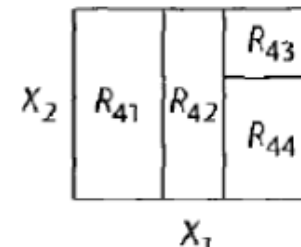
(a)
Branch 1—to 2 Regions
Best split based on X_1



(b)
Branch 2—to 3 Regions
Best split based on X_2 in R_{22}



(c)
Branch 3—to 4 Regions
Best split based on X_1 in R_{31}



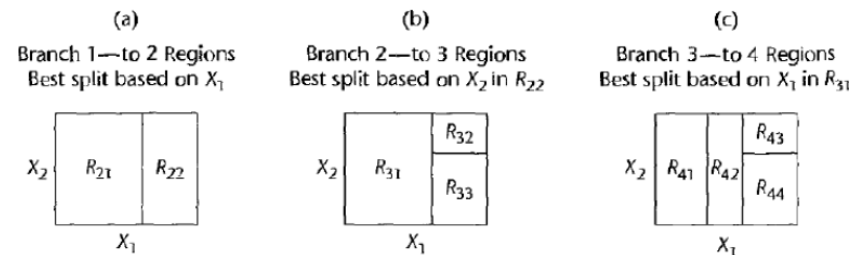
Regression Trees, cont'd

We now elaborate on Step 1 above. How do we construct the regions $R_1, \dots, R_J$?

- In theory, the regions could have any shape. However, we choose to divide the predictor space into high-dimensional rectangles, or boxes, for simplicity and for ease of interpretation of the resulting predictive model. The goal is to find boxes $R_1, \dots, R_J$ that minimize the RSS, given by

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

- where $\bar{y}_{R_j}$ is the mean response for the training observations within the j th box. Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes. For this reason, we take a top-down, greedy approach that is known as recursive binary splitting. The approach is top-down because it begins at the top of the tree (at which point all observations belong to a single region) and then successively splits the predictor space; each split is indicated via two new branches further down on the tree. It is greedy because at each step of the tree-building process, the best split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.



R library and Example

To develop the wine rating model, we will use data donated to the UCI Machine Learning Repository (<http://archive.ics.uci.edu/ml>) by P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Their dataset includes examples of red and white Vinho Verde wines from Portugal—one of the world's leading wine-producing countries.

Because the factors that contribute to a highly rated wine may differ between the red and white varieties, for this analysis we will examine only the more popular white wines.

Regression trees syntax

using the `rpart()` function in the `rpart` package

Building the model:

```
m <- rpart(dv ~ iv, data = mydata)
```

- `dv` is the dependent variable in the `mydata` data frame to be modeled
- `iv` is an R formula specifying the independent variables in the `mydata` data frame to use in the model
- `data` specifies the data frame in which the `dv` and `iv` variables can be found

The function will return a regression tree model object that can be used to make predictions.

Making predictions:

```
p <- predict(m, test, type = "vector")
```

- `m` is a model trained by the `rpart()` function
- `test` is a data frame containing test data with the same features as the training data used to build the model
- `type` specifies the type of prediction to return, either `"vector"` (for predicted numeric values), `"class"` for predicted classes, or `"prob"` (for predicted class probabilities)

The function will return a vector of predictions depending on the `type` parameter.

Example:

```
wine_model <- rpart(quality ~ alcohol + sulfates,  
                    data = wine_train)
```

```
wine_predictions <- predict(wine_model, wine_test)
```

Understanding Ensembles

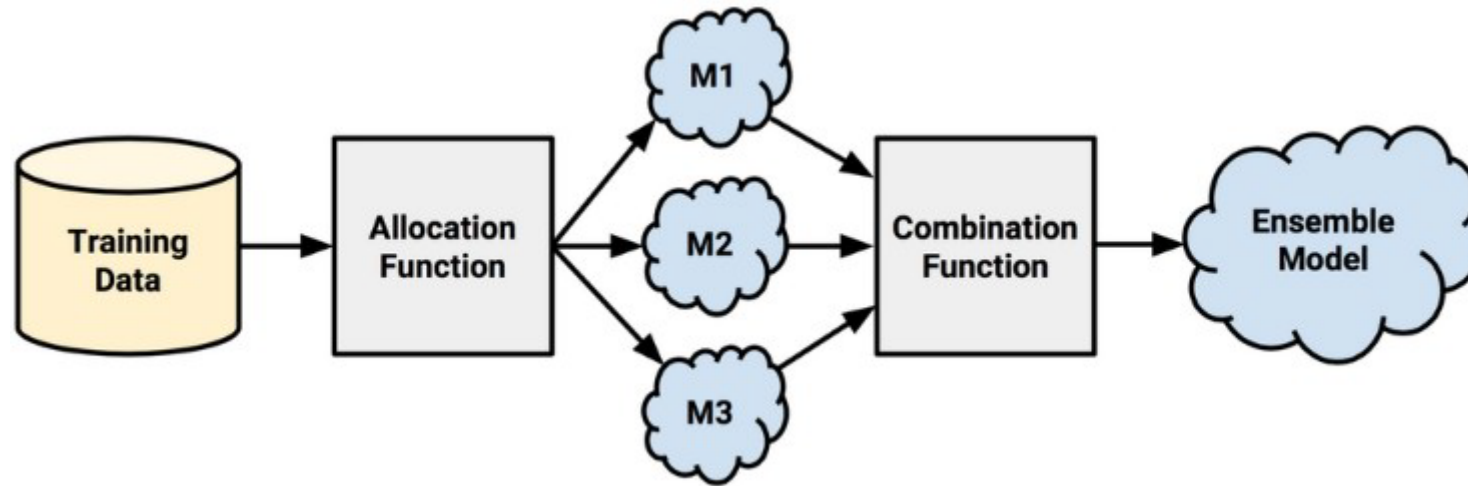
Suppose you were a contestant on a television trivia show that allowed you to choose a panel of five friends to assist you with answering the final question for the million-dollar prize. Most people would try to stack the panel with a diverse set of subject matter experts. A panel containing professors of literature, science, history, and art, along with a current pop-culture expert would be safely well rounded. Given their breadth of knowledge, it would be unlikely that a question would stump the group.

The meta-learning approach that utilizes a similar principle of creating a varied team of experts is known as an ensemble. All ensemble methods are based on the idea that by combining multiple weaker learners, a stronger learner is created. The various ensemble methods can be distinguished, in large part, by the answers to two questions:

- How are the weak learning models chosen and/or constructed?
- How are the weak learners' predictions combined to make a single final prediction?

Understanding Ensembles, *cont'd*

When answering these questions, it can be helpful to imagine the ensemble in terms of the following process diagram; nearly all ensemble approaches follow this pattern:



Understanding Ensembles, *cont'd*

First, input training data is used to build a number of models. The allocation function dictates how much of the training data each model receives. Do they each receive the full training dataset or merely a sample? Do they each receive every feature or a subset?

Although the ideal ensemble includes a diverse set of models, the allocation function can increase diversity by artificially varying the input data to bias the resulting learners, even if they are the same type. For instance, in an ensemble of decision trees, the allocation function might use bootstrap sampling to construct unique training datasets for each tree, or it may pass each one a different subset of features. On the other hand, if the ensemble already includes a diverse set of algorithms—such as a neural network, a decision tree, and a k-NN classifier—then the allocation function might pass the data on to each algorithm relatively unchanged.

After the ensemble's models are constructed, they can be used to generate a set of predictions, which must be managed in some way. The combination function governs how disagreements among the predictions are reconciled. For example, the ensemble might use a majority vote to determine the final prediction, or it could use a more complex strategy such as weighting each model's votes based on its prior performance.

Some ensembles even utilize another model to learn a combination function from various combinations of predictions. For example, suppose that when M_1 and M_2 both vote yes, the actual class value is usually no. In this case, the ensemble could learn to ignore the vote of M_1 and M_2 when they agree. This process of using the predictions of several models to train a final arbiter model is known as stacking.

Understanding Ensembles, *cont'd*

One of the benefits of using ensembles is that they may allow you to spend less time in pursuit of a single best model. Instead, you can train a number of reasonably strong candidates and combine them. Yet convenience isn't the only reason why ensemble-based methods continue to rack up wins in machine learning competitions; ensembles also offer a number of performance advantages over single models:

- **Better generalizability to future problems:** As the opinions of several learners are incorporated into a single final prediction, no single bias is able to dominate. This reduces the chance of overfitting to a learning task.
- **Improved performance on massive or miniscule datasets:** Many models run into memory or complexity limits when an extremely large set of features or examples are used, making it more efficient to train several small models than a single full model. Conversely, ensembles also do well on the smallest datasets because resampling methods like bootstrapping are inherently part of many ensemble designs. Perhaps most importantly, it is often possible to train an ensemble in parallel using distributed computing methods.
- **The ability to synthesize data from distinct domains:** Since there is no one-size-fits-all learning algorithm, the ensemble's ability to incorporate evidence from multiple types of learners is increasingly important as complex phenomena rely on data drawn from diverse domains.
- **A more nuanced understanding of difficult learning tasks:** Real-world phenomena are often extremely complex, with many interacting intricacies. Models that divide the task into smaller portions are likely to more accurately capture subtle patterns that a single global model might miss.

None of these benefits would be very helpful if you weren't able to easily apply ensemble methods in R, and there are many packages available to do just that.

Bagging on Decision Trees

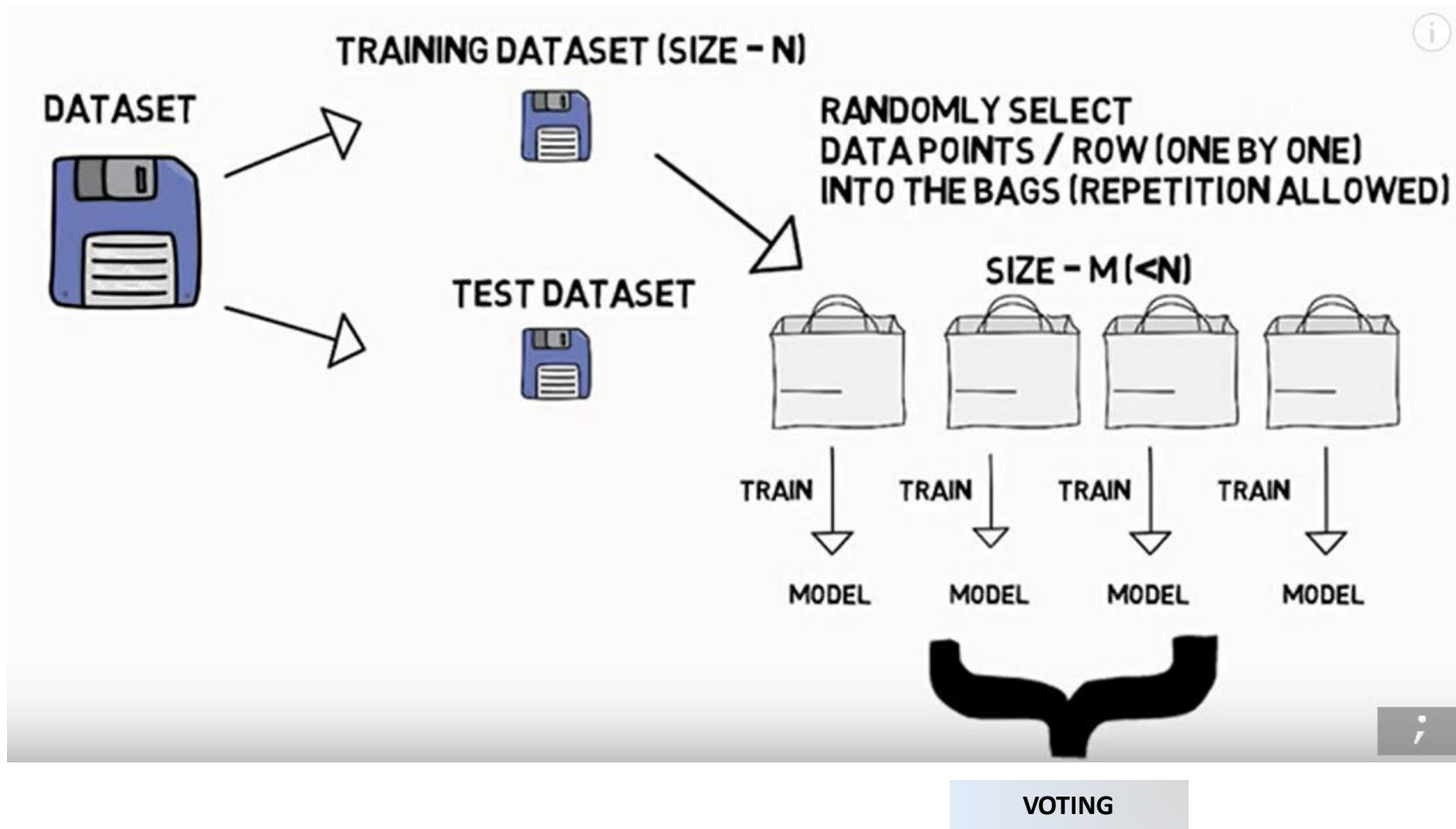
One of the first ensemble methods to gain widespread acceptance used a technique called **bootstrap aggregating**, or **bagging** for short. As described by Leo Breiman in 1994, bagging generates a number of training datasets by bootstrap sampling the original training data. These datasets are then used to generate a set of models using a single learning algorithm. The models' predictions are combined using voting (for classification) or averaging (for numeric prediction).

Bagging on Decision Trees, cont'd

By averaging a collection of bootstrap samples from the training data set, we can dramatically reduce the model variance of trees, leading to improved prediction. This is called **bagging**.

- Bagging = Bootstrap Aggregating
- For a given B , generate B bootstrapped training data sets.
- For each test observation, we record the class predicted by each of B decision trees, and choose the majority vote.
- The overall prediction is the most commonly occurring class(majority vote) among the B predictions.

Bagging on Decision Trees, cont'd



Boosting

Another common ensemble-based method is called **boosting**, because it boosts the performance of weak learners to attain the performance of stronger learners.

This method is based largely on the work of Robert Schapire and Yoav Freund, who have published extensively on the topic.

Similar to bagging, boosting uses ensembles of models trained on resampled data and a vote to determine the final prediction. There are two key distinctions. First, the resampled datasets in boosting are constructed specifically to generate complementary learners. Second, rather than giving each learner an equal vote, boosting gives each learner's vote a weight based on its past performance. Models that perform better have greater influence over the ensemble's final prediction.

Boosting will result in performance that is often better and certainly no worse than the best of the models in the ensemble. Since the models in the ensemble are built to be complementary, it is possible to increase ensemble performance to an arbitrary threshold simply by adding additional classifiers to the group, assuming that each additional classifier performs better than random chance. Given the obvious utility of this finding, boosting is thought to be one of the most significant discoveries in machine learning.

Boosting, *cont'd*

A boosting algorithm called **AdaBoost**, or **adaptive boosting**, was proposed by Freund and Schapire in 1997. The algorithm is based on the idea of generating weak learners that iteratively learn a larger portion of the difficult-to-classify examples in the training data by paying more attention (that is, giving more weight) to often misclassified examples.

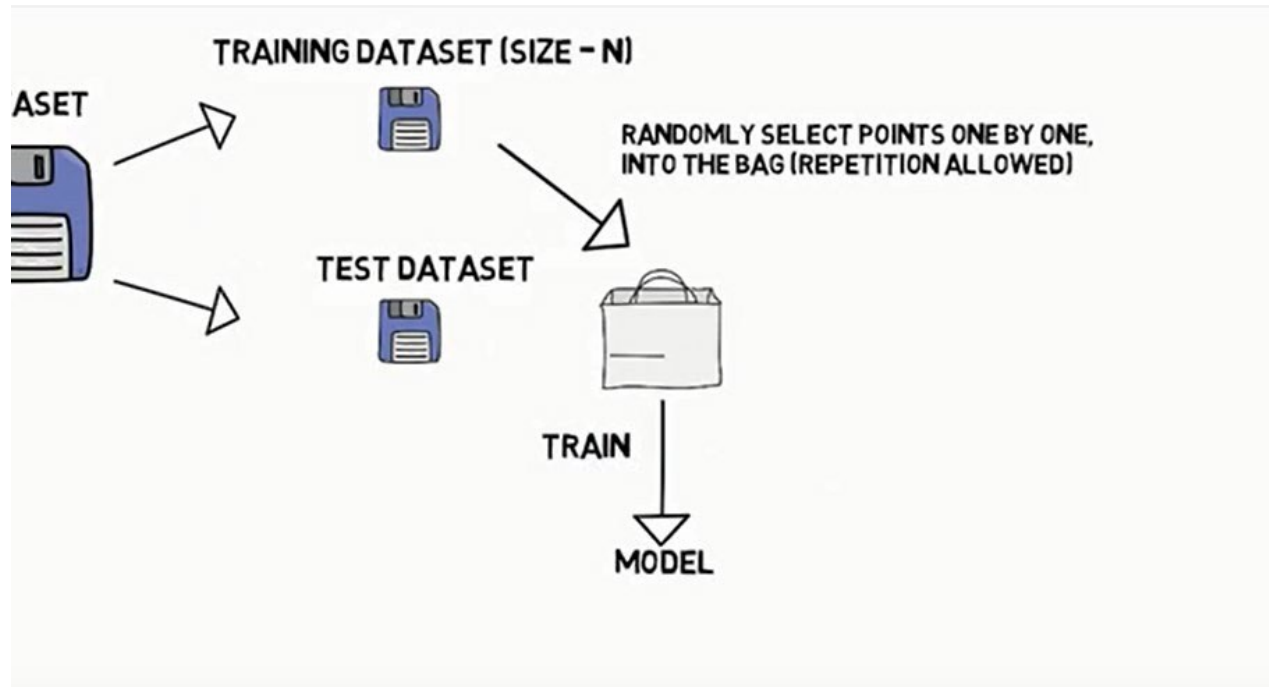
Beginning from an unweighted dataset, the first classifier attempts to model the outcome. Examples that the classifier predicted correctly will be less likely to appear in the training dataset for the following classifier, and conversely, the difficult-to-classify examples will appear more frequently.

As additional rounds of weak learners are added, they are trained on data with successively more difficult examples. The process continues until the desired overall error rate is reached or performance no longer improves. At that point, each classifier's vote is weighted according to its accuracy on the training data on which it was built.

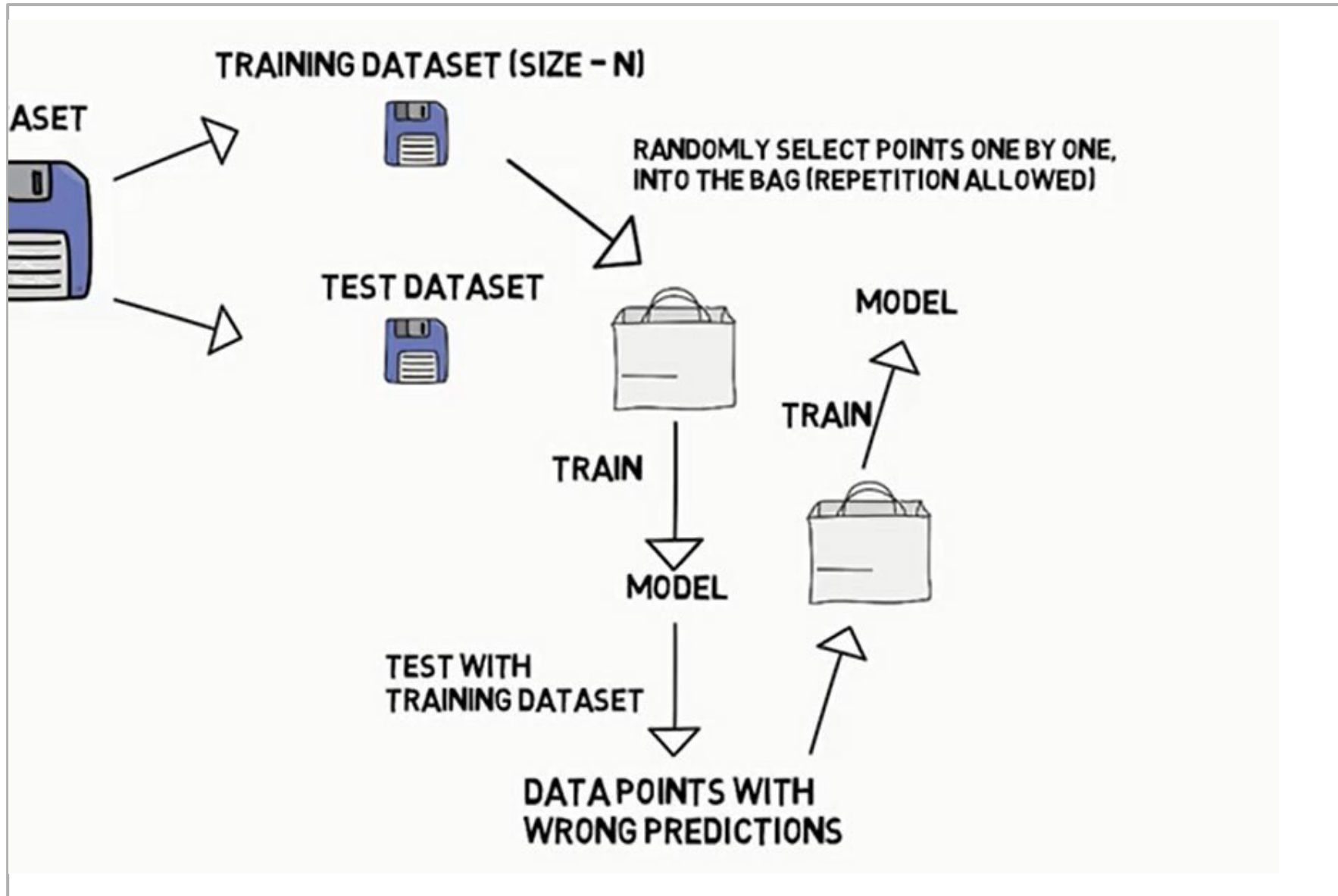
Boosting, *cont'd*

- Boosting is little variation from Bagging
- Gives Emphasis on selecting points which give wrong predictions

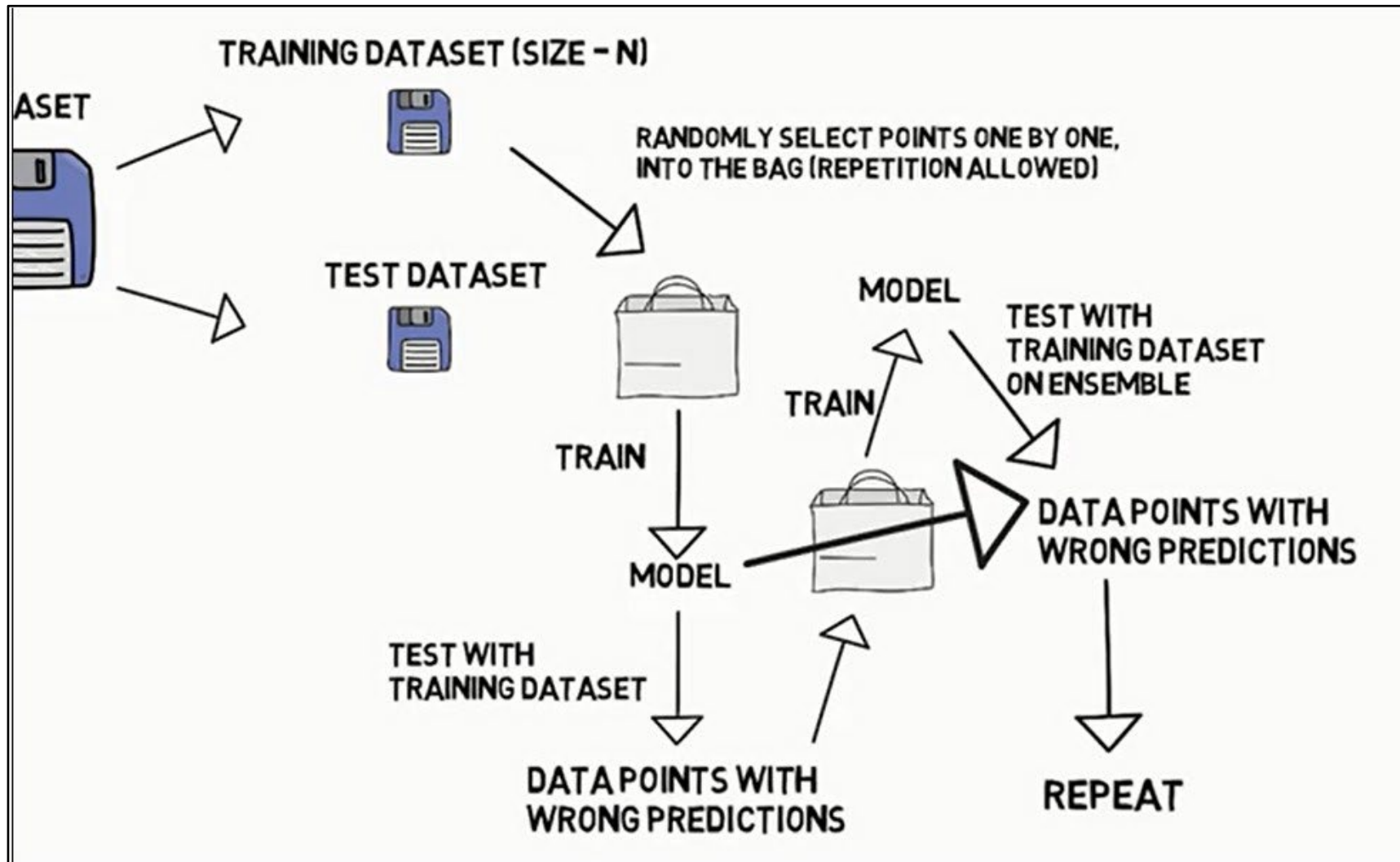
Example:



Boosting, *cont'd*



Boosting, *cont'd*



This results in huge accuracy but also tends to overfit.
Anyways, this is generally a better technique!!

Random Forest

Another ensemble-based method called random forests (or decision tree forests) focuses only on ensembles of decision trees. This method was championed by Leo Breiman and Adele Cutler, and combines the base principles of bagging with random feature selection to add additional diversity to the decision tree models. After the ensemble of trees (the forest) is generated, the model uses a vote to combine the trees' predictions.

- The idea for bagging of decision trees is to average many noisy trees and hence to reduce the model variance.
- However, since each tree generated in bagging is based on identical features, the expectation value of the averages is the same as the expectation of any one of them. This means the bias will not be improved.
- Random forests is a modification of bagging that builds a large collection of de-correlated trees, and then averages them.

Random Forest, *cont'd*

Strengths	Weaknesses
• An all-purpose model that performs well on most problems • Can handle noisy or missing data, as well as categorical or continuous features • Selects only the most important features • Can be used on data with an extremely large number of features or examples	• Unlike a decision tree, the model is not easily interpretable

Random Forest, *cont'd*

Random forest syntax

using the `randomForest()` function in the `randomForest` package

Building the classifier:

```
m <- randomForest(train, class, ntree = 500, mtry = sqrt(p))
```

- `train` is a data frame containing training data
- `class` is a factor vector with the class for each row in the training data
- `ntree` is an integer specifying the number of trees to grow
- `mtry` is an optional integer specifying the number of features to randomly select at each split (uses `sqrt(p)` by default, where `p` is the number of features in the data)

The function will return a random forest object that can be used to make predictions.

Making predictions:

```
p <- predict(m, test, type = "response")
```

- `m` is a model trained by the `randomForest()` function
- `test` is a data frame containing test data with the same features as the training data used to build the classifier
- `type` is either `"response"`, `"prob"`, or `"votes"` and is used to indicate whether the predictions vector should contain the predicted class, the predicted probabilities, or a matrix of vote counts, respectively.

The function will return predictions according to the value of the `type` parameter.

Example:

```
credit_model <- randomForest(credit_train, loan_default)
credit_prediction <- predict(credit_model, credit_test)
```

Measures

The kappa statistic:

Kappa values range from zero to a maximum of one, which indicates perfect agreement between the model's predictions and the true values. Values less than one indicate imperfect agreement. Depending on how a model is to be used, the interpretation of the kappa statistic might vary. One common interpretation is shown as follows:

- Poor agreement = less than 0.20
- Fair agreement = 0.20 to 0.40
- Moderate agreement = 0.40 to 0.60
- Good agreement = 0.60 to 0.80
- Very good agreement = 0.80 to 1.00

|

It's important to note that these categories are subjective. While "good agreement" may be more than adequate for predicting someone's favorite ice cream flavor, "very good agreement" may not suffice if your goal is to identify birth defects.

Measures, *cont'd*

Sensitivity and specificity:

The **sensitivity** of a model (also called the true positive rate), measures the proportion of positive examples that were correctly classified. Therefore, as shown in the following formula, it is calculated as the number of true positives divided by the total number of positives, both those correctly classified (the true positives), as well as those incorrectly classified (the false negatives):

$$\text{sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The **specificity** of a model (also called the true negative rate), measures the proportion of negative examples that were correctly classified. As with sensitivity, this is computed as the number of true negatives divided by the total number of negatives—the true negatives plus the false positives.

$$\text{specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Measures, *cont'd*

Precision and recall:

The **precision** (also known as the positive predictive value) is defined as the proportion of positive examples that are truly positive; in other words, when a model predicts the positive class, how often is it correct? A precise model will only predict the positive class in cases very likely to be positive. It will be very trustworthy.

Consider what would happen if the model was very imprecise. Over time, the results would be less likely to be trusted. In the context of information retrieval, this would be similar to a search engine such as Google returning unrelated results. Eventually, users would switch to a competitor like Bing.

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

On the other hand, **recall** is a measure of how complete the results are. As shown in the following formula, this is defined as the number of true positives over the total number of positives. You may have already recognized this as the same as sensitivity; however, the interpretation differs slightly.

A model with high recall captures a large portion of the positive examples, meaning that it has wide breadth. For example, a search engine with high recall returns a large number of documents pertinent to the search query.

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Measures, *cont'd*

The F-measure:

A measure of model performance that combines precision and recall into a single number is known as the **F-measure** (also sometimes called the **F1** score or the F-score). The F-measure combines precision and recall using the harmonic mean, a type of average that is used for rates of change. The harmonic mean is used rather than the more common arithmetic mean since both precision and recall are expressed as proportions between zero and one, which can be interpreted as rates. The following is the formula for the F-measure:

$$\text{F - measure} = \frac{2 \times \text{precision} \times \text{recall}}{\text{recall} + \text{precision}} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}}$$

Since the F-measure describes model performance in a single number, it provides a convenient way to compare several models side-by-side. However, this assumes that equal weight should be assigned to precision and recall, an assumption that is not always valid. It is possible to calculate F-scores using different weights for precision and recall, but choosing the weights can be tricky at best and arbitrary at worst. A better practice is to use measures such as the F-score in combination with methods that consider a model's strengths and weaknesses more globally, such as those described in the next section.